



## Examen 2

A continuación, se presentan 4 escenarios. Para cada escenario, debes diseñar e implementar la función solicitada utilizando como soporte las herramientas provistas en nuestra biblioteca de grafos.

El objetivo es analizar el problema para poder elegir el o los mejores algoritmos para poder estructurar una solución. Debes mantener las firmas de las funciones exactamente como se te proporcionan en la plantilla del código.

### Los parámetros de evaluación estarán comprendidos por:

1. Correcta ejecución y desarrollo de los ejercicios que darán un total de 20 puntos.
2. El ejercicio es bueno o malo, si pasa todos los test, pasa a revisión con un preparador y en caso de visto bueno, se obtendrán los puntos respectivos del ejercicio. De manera que es suficiente con que un solo test falle para que no se otorgue ninguna calificación al ejercicio.

### Aspectos a tomar en cuenta:

- Se tendrán 2 horas para resolver la evaluación.
- Para ser evaluados deberán llamar a alguno de los preparadores para mostrar el código y la ejecución del o los ejercicios realizado(s) para su corrección.
- Tienen intentos infinitos para recibir correcciones mientras estén en el rango de las 2 horas dadas. (tomar en consideración que se puede formar una “cola”, y pueden tardar en ser atendidos)
- Está prohibido el uso de internet, inteligencia artificial y/o ayudarse entre los compañeros. Hacer uso de alguno de estos resultará en la anulación total de la evaluación o incluso podría ser de la materia dependiendo del caso.

¡Éxitos!



## Ejercicios

### 1) El Evento Principal (Clasificación de Luchadores) [6 pts]

Eres el gerente general de una empresa de lucha libre profesional. Tienes a tu disposición un listado completo de luchadores y un registro definitivo de todas las rivalidades activas entre ellos. Para organizar tu próximo gran evento, necesitas dividir el vestuario en dos facciones distintas: los "Faces" (los héroes) y los "Heels" (los villanos).

La regla de oro de tu empresa es que cada rivalidad debe enfrentar estrictamente a un Face contra un Heel. Diseña una función que determine si es lógicamente posible dividir tu listado actual de luchadores en estas dos facciones sin romper la regla de oro. Si es posible, tu algoritmo debe agrupar a los luchadores en sus respectivas colecciones.

### 2) El Atraco [4 pts]

Un ladrón de élite está ejecutando un atraco en una enorme mansión laberíntica. El plano de la mansión está compuesto por intersecciones conectadas por pasillos. El plan del ladrón es absoluto: debe infiltrarse por un Punto de Entrada (Entry), asegurar el botín en la Bóveda (Vault) y escapar por la Ruta de Evacuación (Escape).

El tiempo es vital. Diseña una función que calcule la secuencia continua más corta de intersecciones que el ladrón debe atravesar para completar la infiltración, el robo y la exfiltración. Si la secuencia de eventos es físicamente imposible debido a rutas bloqueadas, la operación debe ser abortada retornando una secuencia vacía.



### 3) Protocolo de Triage de Red [4 pts]

Un evento sísmico ha cortado múltiples enlaces de fibra óptica en una infraestructura global de servidores. Los equipos de reparación están listos para ser desplegados, pero necesitan un plan de acción estructurado.

Tu objetivo es analizar el estado actual de la infraestructura e identificar todas las sub-redes que han quedado completamente aisladas entre sí. Para maximizar el restablecimiento de los servicios, debes generar una lista de estas sub-redes ordenada por prioridad de atención: los equipos deben ser enviados primero a la sub-red que contenga la mayor cantidad de servidores operativos. Si dos sub-redes tienen la misma cantidad de servidores, el orden de empate es irrelevante.



#### 4) Orquestador de Compilación Paralela [6 pts]

Estás desarrollando el sistema de construcción (build system) para un motor gráfico masivo. El código fuente está dividido en cientos de módulos distintos, y muchos de ellos no pueden comenzar a compilar hasta que sus prerequisites estén completamente listos.

Para aprovechar los procesadores multinúcleo modernos, tu sistema debe agrupar los módulos en "Fases de Compilación" que se ejecutarán en paralelo:

- **Fase 0:** Módulos que no requieren ningún otro módulo para compilarse.
- **Fase 1:** Módulos que solo necesitan que la Fase 0 haya terminado.
- **Fase N:** Módulos que dependen de las fases anteriores.

Para asegurar que los logs del sistema sean consistentes, dentro de cada fase, los módulos deben ordenarse estrictamente por orden alfabético. Finalmente, si el sistema detecta que un desarrollador introdujo una dependencia circular (haciendo que el proyecto sea incompilable), el orquestador debe abortar la operación de forma segura y retornar un plan de compilación vacío.